



Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

Differential bond energy algorithm for optimal vertical fragmentation of distributed databases

Shikha Mehta, Parul Agarwal*, Prakhar Shrivastava, Jharna Barlawala

Department of Computer Science and Information Technology, Jaypee Institute of Information Technology, Noida 201301, India

ARTICLE INFO

Article history:

Received 21 March 2018

Revised 14 August 2018

Accepted 24 September 2018

Available online 27 September 2018

Keywords:

Distributed database systems

Vertical fragmentation

Bond energy algorithm

Differential evolution algorithm

ABSTRACT

Distributed database systems are gaining importance due to the production in massive amount of data. The efficacy of such systems is highly dependent upon the design of the system. To increase the effectiveness and efficiency of distributed databases, two processes are mainly employed i.e. fragmentation and allocation. Fragmentations can be vertical or horizontal. This work focuses on vertical fragmentation design methods. In this paper, a novel differential bond energy (DBE) algorithm is proposed with objective to determine optimal partition point. The performance of proposed algorithm is compared with classical bond energy algorithm (BEA) on basis of global affinity measure (GAM) value. Results are depicted in form of line graphs. The mean difference in GAM values for both algorithms are also illustrated. The experimental results portrays that DBE is suitable for vertical fragmentation of high dimensional problems as it attain high GAM value as compared to BEA on various datasets.

© 2018 The Author(s). Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

Looking at the vast amounts of data that is generated every day, there is a great need of an advanced, powerful and effective technology that can handle the storage, processing and analysis of the data efficiently thus saving the hardware resources and time. The delocalization of the data across multiple systems connected through a network greatly reduces the costs and guarantee better availability and performance. A distributed database (DDB) is a group of many logically interrelated databases distributed over a computer network. A distributed database management system (DDBMS) is the software that manages the DDB, and gives an access system that makes this distribution transparent to the people (Rababaah, 2005). Design of distributed database systems is one of the most complex problems which involve Fragmentation and Allocation. Fragmentation divides the data into fragments, which must be allocated to sites over the network in the phase of allocation. The allocation problem performs the optimal distribution of fragments to set F on site set S. The process of fragmentation falls under two categories: Vertical fragmentation and Horizontal fragmentation. Vertical Fragmentation (VF) partitions the relation vertically into disassociated sets of smaller relations while Horizontal Fragmentation (HF) partitions the relation into disassociated tuples.

Vertical Fragmentation and Allocation are NP-hard problems a heuristic approach could not reach the optimal solution. Various nature inspired meta-heuristic algorithms are developed to provide near optimal solutions to NP-Hard problems (Agarwal and Mehta, 2017; Agarwal and Mehta, 2014). A number of algorithms have been proposed by researchers to address vertical fragmentation problem yet an optimized approach is still missing. A classical algorithm given for vertical fragmentation is known as Bond energy algorithm (BEA) (Özsu and Valduriez, 2011). The paper presents an optimized scheme for vertical fragmentation in distributed database system using the Differential Evolution Algorithm (DE). As DE is one of the strongest evolutionary algorithms, there is a possibility of getting better result using this approach. This is the first time when any evolutionary algorithm is integrated with bond energy algorithm to improve vertical fragmentation. The objective of the presented work is to perform optimal vertical fragmentation in distributed databases using differential bond energy algorithm. This algorithm is hybrid of differential evolution (evolutionary algorithm) and bond energy algorithm (classical vertical fragmentation algorithm). The work focuses on improving the quality of fragments generated by the

Vertical Fragmentation and Allocation are NP-hard problems a heuristic approach could not reach the optimal solution. Various nature inspired meta-heuristic algorithms are developed to provide near optimal solutions to NP-Hard problems (Agarwal and Mehta, 2017; Agarwal and Mehta, 2014). A number of algorithms have been proposed by researchers to address vertical fragmentation problem yet an optimized approach is still missing. A classical algorithm given for vertical fragmentation is known as Bond energy algorithm (BEA) (Özsu and Valduriez, 2011). The paper presents an optimized scheme for vertical fragmentation in distributed database system using the Differential Evolution Algorithm (DE). As DE is one of the strongest evolutionary algorithms, there is a possibility of getting better result using this approach. This is the first time when any evolutionary algorithm is integrated with bond energy algorithm to improve vertical fragmentation. The objective of the presented work is to perform optimal vertical fragmentation in distributed databases using differential bond energy algorithm. This algorithm is hybrid of differential evolution (evolutionary algorithm) and bond energy algorithm (classical vertical fragmentation algorithm). The work focuses on improving the quality of fragments generated by the

* Corresponding author.

E-mail address: parul.agarwal@jiit.ac.in (P. Agarwal).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

algorithms and the quality is measured in terms of GAM i.e. global affinity measure. It indicates the closeness of the attributes with each other. Higher value of GAM depicts better affinity between the attributes in fragment. The proposed algorithm may give better performance as it consider larger number of possible optimal solutions as compared to the classical bond energy algorithm.

This paper is divided into following subsections. Section 2 represents related work. Section 3 describes proposed algorithm. Section 4 illustrates experimental setup. Results and its analysis are depicted in section 5. Section 6 presents conclusion of work.

2. Related work

The basic fundamentals of a distributed database system have been highlighted by Haroun Rababaah (2005). The work describes the majority of the research work that has been done in distributed database system. This field of research includes efficient query processing, database design and fragmentation. Pérez et al. (2000) proposed an extension of the DFAR model to solve the problem of vertical fragmentation and allocation, along with considering the capacity constraints of a site. Hui Ma and Schewe (2006) proposed a heuristic approach based cost model to solve the problem of vertical fragmentation and allocation. Pazos et al. (2010) addressed the distributed database design problem by considering the nonlinear nature of roundtrip response time. They proposed a metaheuristic approach and a mathematical zero one integer programming model to minimize the delays in query processing. Khan and Hoque (2010) have proposed a different fragmentation technique for horizontal fragmentation which is successful in fragmenting at an initial stage by using the predetermined information collected during requirement analysis phase. Tosun (2014) has focused upon the data allocation problem and solved it by presenting a hybrid genetic multi-start tabu search algorithm that gives much better results than genetic algorithm, ant colony optimization and tabu search algorithm. Rahimi et al. (2015) have solved the simultaneous vertical fragmentation and allocation problem by modifying the formula of affinity measure in the Classical Bond Energy Algorithm and compared the experimental results of the proposed modified algorithm with Classical Bond Energy Algorithm. Abdel-Raouf (2014) have presented a scheme called the vertical fragmentation, allocation and replication scheme (VFAR) that also fragments the data at initial stage and then allocate and replicate the data at multiple sites using the information collected at requirement analysis. Agarwal and Mehta (2016) proposed a modified flower Pollination algorithm to increase the convergence capability of clustering. Amer et al. (2017) presented an optimized technique for simultaneous horizontal fragmentation and allocation scheme. The algorithm accuracy is measured on basis of transmission costs. Two similar kinds of work by Goli and Rankoohi (2012) and Du et al. (2006) presents the vertical partition techniques. However presented work differs from these in following ways: (see Table 1).

Table 1
Proposed algorithm vs. HACA vs. GRGS-GA.

Parameters	Differential Bond Energy (Proposed Algorithm)	HACA (Goli and Rankoohi ref) (Goli and Rankoohi, 2012)	Group oriented Restricted Growth String GA (GRGS-GA) (Du et al., 2006)
Nature Inspired Algorithm used	Differential Evolution	Ant Colony Optimization	Genetic Algorithm
Algorithm for fragmentation	Bond energy algorithm which is a classical vertical fragmentation algorithm used in distributed databases.	Clustering which is data mining technique.	Clustering which is data mining technique.
Compared Algorithms	Bond Energy Algorithm	Group oriented Restricted Growth String GA (GRGS-GA)	RanDom integer string GA (RD-GA); restricted growth string (RGS)
Performance Evaluation	Quality of fragments- measured in terms of global affinity measure (GAM)	average cost of transaction processing	Average partition evaluator values of final partition

Both HACA (Goli and Rankoohi, 2012) and GRGS-GA (Du et al., 2006) evaluate the performance of Ant colony and Genetic Algorithm respectively rather than the quality of fragments generated. Besides, most of the researchers have used a heuristic approach to obtain the most optimal solution. The difference exists is in between the types of different heuristic methods they have used. These are mathematical cost model, modified Bond Energy Algorithm, tabu search algorithm and many more. Very few of the existing algorithms used the metaheuristic approach for vertical fragmentation. The advantage of proposed algorithm i.e. combining the differential evolution algorithm with the classical bond energy algorithm is that it checks for all the possible combination of solutions as compared to classical BEA and hence has more probability to give better values of Global Affinity Measure (GAM). Higher GAM value reveals that DBE algorithm fragments the attributes of a database more effectively.

3. Proposed algorithm

3.1. Bond energy algorithm (BEA)

Bond Energy Algorithm was proposed by McCormick et al. in 1972 and is used for the purpose of grouping of attributes of a relation on the basis of closeness between the attributes (Özsu and Valduriez, 2011). The various parameters of algorithms are described as below: (see Table 2).

Let $R(A_1, A_2, \dots, A_n)$ be the relation into consideration where A_1, A_2 are the attributes of the relation R . Q be the set of user queries defined as $\{q_1, q_2, \dots, q_m\}$. S be the total number of sites. The Attribute Usage Value, $auv(q_i, A_j)$ is defined as follows:

$$auv(q_i, A_j) = \begin{cases} 1, & \text{if } A_j \text{ is used by } q_i \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

The Attribute Affinity Measure between any two attributes, $aam(A_i, A_j)$ that determines the closeness between the two attributes on basis of the queries accessed by the sites. It can be defined as:

$$aam(A_i, A_j) = \sum_{k: auv(q_k, A_i)=1 \wedge auv(q_k, A_j)=1} \sum_{\forall S_l} naa_l(q_k) aafm_l(q_k) \quad (2)$$

where $naa_l(q_k)$ = number of accesses to attributes (A_i, A_j) for each execution of query q_k at site S_l and

$aafm_l(q_k)$ = number of times the query q_k is accessed at site S_l .

The BEA takes the Attribute Affinity Matrix as input and then outputs the clustered Affinity Matrix. The attributes are permuted with the objective to maximize the value of Global Affinity Measure (GAM). The GAM value indicates that how close the attributes with greater affinities with each other are grouped.

$$GAM = \sum_i^n \sum_{j=1}^n aam(A_i, A_j) * (aam(A_i, A_{j-1}) + aam(A_i, A_{j+1})) \quad (3)$$

Table 2

Notations Meaning.

Notations	Meanings
R	Relation into consideration
A _i	Attributes of Relation R
Q	Set of Queries
S	Total number of sites
auv	Attribute usage value
aam	Attribute affinity measure
naa ₁ (q _k)	Number of access to attributes for each execution of query q _k at site s _i
aafm _i (q _k)	Number of times the query (q _k) is accessed at site s _i
GAM	Global affinity measure
AA	Attribute affinity matrix
CA	Clustered affinity matrix
cntr (A _i , A _k , A _j)	Net contribution made by placing A _k between A _i and A _j
b (A _x , A _y)	Bond between two attribute A _x and A _y
TL	Attribute after partition point
BR	Attribute before partition point
Cr	Crossover rate
F	Scale factor
P	Population Size
g	Number of generations
n	Number of attributes
TQ	group of queries that access only TL
BQ	group of queries that access only BR
OQ	group of queries that access both TL and BR
CTQ	total number of accesses to attributes by queries that access only TL
CBQ	total number of accesses to attributes by queries that access only BR
COQ	total number of accesses to attributes by queries that access both TL and BR

where $aam(A_0, A_j) = aam(A_i, A_0) = aam(A_{n+1}, A_j) = aam(A_i, A_{n+1}) = 0$

Clustering steps of bond energy algorithm (Özsu and Valduriez, 2011) is defined in Algorithm 1.

Algorithm 1 (Bond Energy Algorithm (BEA)).

Input: The Attribute Affinity matrix (AA)
Output: The clustered affinity matrix (CA) which is a perturbation of AA
STEP A. Initialization: Place and fix one of the columns of AA in CA.
STEP B. Iteration: Place each of the remaining columns in the remaining positions in the CA matrix. For each column, choose the placement that makes the most contribution to the global affinity measure. Continue until all columns are placed.
The net contribution to GAM is made by placing attribute A_k between A_i and A_j is:

$$cntr(A_i, A_k, A_j) = 2b(A_i, A_k) + 2b(A_k, A_j) - 2b(A_i, A_j) \quad (4)$$
where $b(A_x, A_y)$ = bond between 2 attributes A_x and A_y and it can be calculated as:

$$b(A_x, A_y) = \sum_{z=1}^n aam(A_z, A_x) aam(A_z, A_y) \quad (5)$$
STEP C. Row order: Order the rows according to the column ordering.

Partition point is the point that divides the attributes in different groups so that they can be allocated to the multiple sites. 2-way partitioning has been performed i.e. fragment the attributes in two groups have to be allocated to two sites.

To find the partition point (Özsu and Valduriez, 2011), let, TL be Cluster-1 (Attributes before the partition point in CA)
BR be Cluster-2 (Attributes after the partition point in CA).
The point along the diagonal that has the maximum value for $CTQ * CBQ - COQ^2$ is the partition point. Attributes on left of the par-

tion point belongs to one site while the attributes on right side belongs to another site.

3.2. Differential Bond energy algorithm (DBE)

Differential bond energy algorithm (DBE) is proposed by incorporating evolutionary mechanisms of differential evolution algorithm (DE) in classical bond energy algorithm (BEA). The main idea of inculcating DE (Storn and Price, 1997) in BEA is that due to random population generation, BEA gets more combinations of attributes to find optimal ordering in clustering affinity matrix. Mutation and crossover strategies of DE could improve the clustering efficacy. DE is applied to explore different permutations of the attributes in AA so as to give maximum value of GAM. The permutation of the best solution gives the optimal Clustered Affinity Matrix (CA). Details of proposed algorithm are given in Algorithm 2.

Algorithm 2 (Differential Bond Energy Algorithm (DBE)).

Input: The Attribute Affinity matrix (AA)
Output: The clustered affinity matrix CA which is a perturbation of AA
STEP A. Initialize the Control parameters:
Crossover rate, CR = 0.5 (0 < CR < 1)
Scale factor, F = 1.8 (0 < F < 2)
(The values of F and CR have been selected as 1.8 and 0.5 respectively because the experimental studies have proved that using these values provide the best result.)
Initialize the population size 'p' and number of generations (g) and limit where limit is the stopping parameter for the algorithm. The algorithm stops if the solution does not improve continuously for 'limit' number of times.
STEP B. Initialize the population.
(Each vector is a randomly generated possible permutation of the given input matrix.)
Example: If number of attributes (n) = 5, then
X1 = [2,4,0,3,1],
X2 = [1,3,0,4,2], and so on.
STEP C. Calculate fitness value for each vector.
Fitness (Xi) = Global Affinity Measure (GAM) corresponding to the permutation in X[i] according to the Eq. (3)
STEP D. Memorize the vector with the best fitness value.
STEP E. gen = 0, count = 0
STEP F. while gen < generations and count < limit:
for each Population vector i:
//Mutation (rand/1/bin)
Sort V and map the indices to the sorted array.
This is our mutated vector.
//Crossover
Apply crossover operator between X[i] and V and store the offspring in U.
The crossover operator is influenced by the Liuhai Crossover (Mei et al., 2010) focusing on the distance between the positions of indices of maximum attribute affinity value in AA.
//Selection
Now, we select the vector among X[i] and U based on the maximum fitness value.
Update the best fitness vector.
If vector is updated, count = 0 else count+=1
If (count == limit) break out of the loop.
STEP G. Print the matrix and the GAM value corresponding to the best order at the end.

Table 3
Datasets description and corresponding query files.

S. No.	Title	No. of attributes	No. of queries in file
1	Iris	6	30
2	Wine Reviews	10	50
3	Events	22	110
4	Mental Health Survey	27	135
5	Breast Cancer	32	160
6	Sugar concentration	46	230
7	Corporate Market Analysis	56	280
8	Specific Languages	64	320
9	Global Terrorism	135	640
10	Stack Overflow dataset	164	616
11	Pensions	274	1370

Table 4
Combinations of Population size and No. of generations.

Population size 'p'	No. of generations 'g'
10	100
10	300
25	100
25	300
50	100
50	300

4. Experimental setup

The DBE algorithm is implemented in Python 3, with the hardware configurations of the computer system as 1.6 GHz Intel Core i5 processor, 8 GB 1600 MHz DDR3 RAM and Mac OS High Sierra Operating System. The execution of proposed algorithm is made using query files for different datasets. There are 11 real datasets that are collected from (<https://www.kaggle.com/datasets>). The query files were randomly generated. The sample query files contain the queries run on a site and the attribute names corresponding to a particular dataset. These sample query files are used to create the attribute affinity matrix (AA) using Eq. (2). AA is required as an input by both the algorithms i.e. BEA and DBE.

The information regarding the datasets and their corresponding query files are shown in Table 3:

The Global Affinity Measure (GAM) is calculated by BEA and DBE algorithm for different datasets. The datasets with different number of attributes 'n' are considered along with varying population size 'p' and number of generations 'g' as shown in Table 4.

5. Results and analysis

Bond energy algorithm (BEA) and proposed algorithm (DBE) is executed on 11 different datasets. These datasets vary in schema as shown in Table 3. Global Affinity Measure (GAM) of Cluster Affinity Matrix (CA) produced by BEA and DBE algorithm has been plotted on the line graphs against the number of attributes 'n'. Figs. 1–6 shows the graphs having different combination of size of initial population 'p' and number of generations 'g'.

The performance of both the algorithms has been measured on the basis of Global Affinity Measure (GAM). The algorithm that gives Maximum Global Affinity Measure is termed as the better algorithm. It has been observed from Figs. 1–6 that as population size increases, DBE tends to give better GAM. This is because increasing population size adds more exploration capability in the algorithm. Exploiting the good explored solutions may increase the possibility of getting optimal solution. Increasing number of generations may also lead towards better solutions but may stagnate at a point of time. After an in-depth analysis of graphs, it can be concluded that for the larger values of 'n', the proposed algorithm gives better results than the traditional BEA. On the

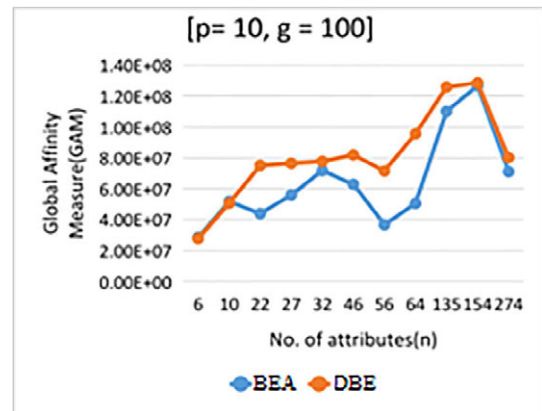


Fig. 1. GAM vs. n (Population Size = 10, No. of Generations = 100).

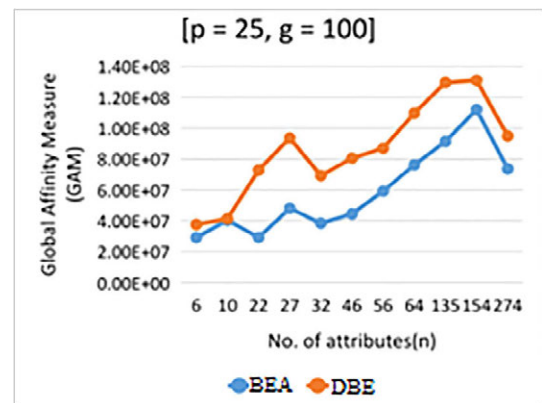


Fig. 2. GAM vs. n (Population Size = 25, No. of Generations = 100).

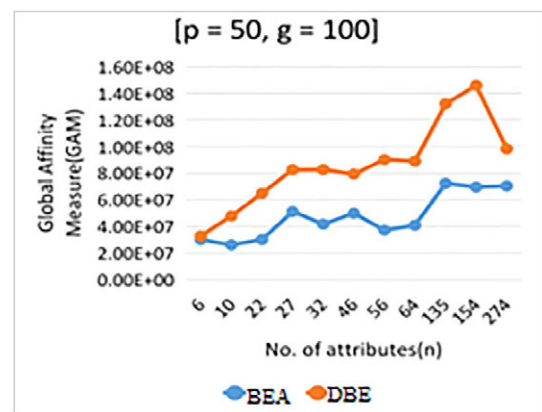


Fig. 3. GAM vs. n (Population Size = 50, No. of Generations = 100).

other side of the spectrum i.e. for small number of attributes, the proposed algorithm DBE performs nearly same to classical bond energy Algorithm. Hence, the proposed algorithm suits the vertical fragmentation process of large attributes. It has also been noticed that as the values of 'p' and 'g' increases, the difference between the values of GAM obtained by DBE algorithm and BEA increases. This is because the proposed algorithm tests a larger number of possible solutions as compared to the classic BEA and hence gives better GAM value in most of the cases.

In order to substantiate the obtained results, the performance analysis of proposed algorithm is made on basis of difference in

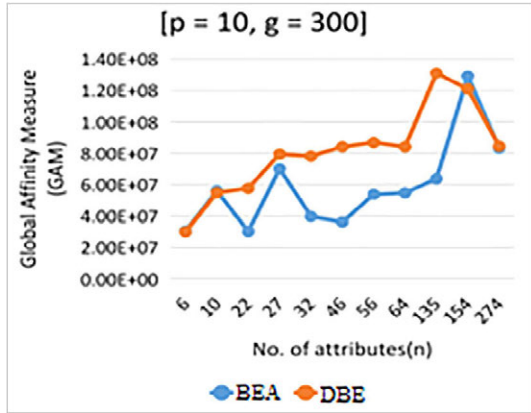


Fig. 4. GAM vs. n (Population Size = 10, No. of Generations = 300).

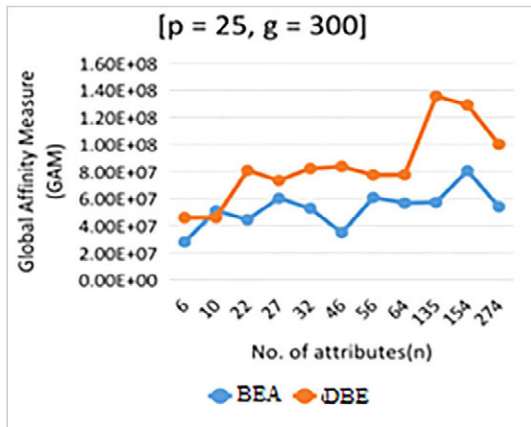


Fig. 5. GAM vs. n (Population Size = 25, No. of Generations = 300).

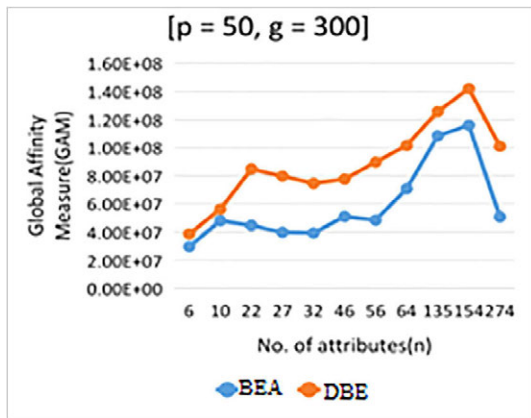


Fig. 6. GAM vs. n (Population Size = 50, No. of Generations = 300).

GAM values obtained. The graph has been plotted between differences of Global Affinity Measure (GAM) of Cluster Affinity Matrix (CAM) obtained from BEA and DBE Algorithm. Figs. 7 and 8 depicts the graph between the difference in GAM values by both algorithms vs. number of attributes 'n' with values of 'g' as 100 and 300 respectively.

The Eq. (6) has been used for calculating the difference is:

$$Diff = GAM_{DBE} - GAM_{BEA} \quad (6)$$

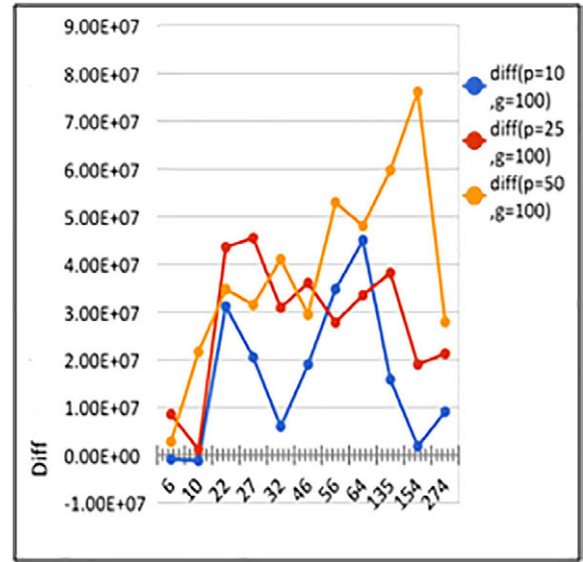


Fig. 7. Graph between the difference in GAM vs. 'n' with g = 100.

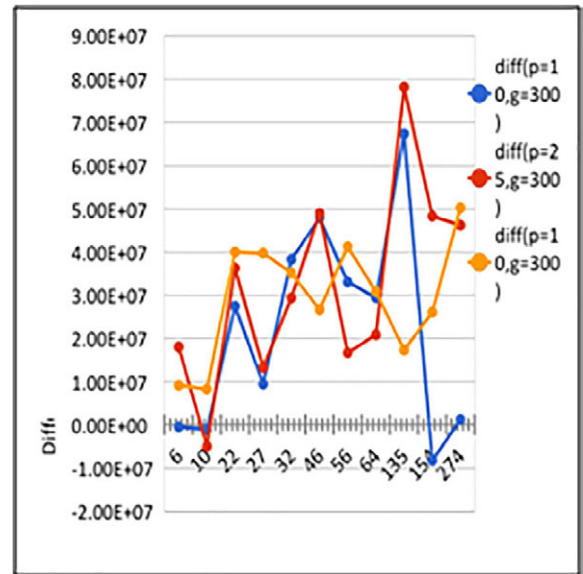


Fig. 8. Graph between the difference in GAM vs. 'n' with g = 300.

In Fig. 7, highest difference is portrayed at 154 dimensional dataset. This signifies that DBE outperform BEA at high dimension dataset. However, the difference is almost negligible for 6–10 attributes for population size 10. In Fig. 8 with 300 generations, the difference is also highest at 154 attribute dataset. It is worth noticeable that difference is highest for population size 25. However, with small population size (p = 10), difference is negative on same attribute. At small dimensions BEA and DBE gives nearly same performance. It has been observed from graphs plotted in Figs. 7 and 8 that most of the part lies above x axis at large population. This implies that DBE algorithm portrays better performance in that area.

For the sake of depicting the effectiveness of overall results, the mean difference between GAM_{DBE} and GAM_{BEA} for each dataset corresponding to each combination of 'p' and 'g' is calculated as follows:

$$Diff_{mean} = \frac{\sum_{i=1}^6 Diff_i}{total\ combination\ (i.e.6)} \quad (7)$$

Table 5

Mean difference of GAM for each dataset.

Title	No. of attributes	Diff _{mean} (in10 ⁶)
Iris	6	6.189
Wine Reviews	10	3.965
Events	22	35.536
Mental Health Survey	27	26.661
Breast Cancer	32	30.123
Sugar concentration	46	34.692
Corporate Market Analysis	56	34.423
Specific Languages	64	34.631
Global Terrorism	135	46.092
Stack Overflow dataset	164	27.213
Pensions	274	25.996

It can be deduced from Table 5 that mean difference in GAM values of DBE and BEA increases with increase in number of attributes. In case of stack overflow and pensions datasets, the difference reduces. However, difference is positive which proves DBE gives better performance and finds good quality fragments as compared to classical BEA algorithm.

Since the difference between GAM_{DBE} and GAM_{BEA} is highly positive in all cases of Table 5, it can be deduced that best values of GAM are obtained from DBE algorithm and the maximum mean difference between GAM_{DBE} and GAM_{BEA} is observed for n = 135 with the value of 46091719.

6. Conclusion

The work focuses on determining optimal partition point of vertical fragmentation in distributed database design. A Differential Bond Energy Algorithm (DBE) has been proposed with the objective to optimize data fragmentation in high dimensional datasets. The experimental results are illustrated in form of Global Affinity Measure. The performance of proposed algorithm has been compared with classical Bond Energy Algorithm (BEA) and graphs have been plotted for the same. The calculated mean difference between GAM_{DBE} and GAM_{BEA} are all positive which shows that DBE algorithm gives better results than the classical BEA. The maximum mean difference between GAM_{DBE} and GAM_{BEA} is observed for dataset with 135 attributes. As the DBE algorithm offers a larger number of possible solutions as compared to the classical BEA, former depicts better GAM value on majority of datasets. Hence, results show that the proposed algorithm is apt for fragmentation

of high dimensional datasets and finds good quality fragments in vertical fragmentation of distributed database design.

References

- Abdel-Raouf, O., 2014. A new hybrid flower pollination algorithm for solving constrained global optimization problems. *Int. J. Appl. Oper. Res.* 4, 1–13.
- Agarwal, P., Mehta, S., 2017. Empirical analysis of five nature-inspired algorithms on real parameter optimization problems. *Artif. Intell. Rev.*, 1–57 <https://doi.org/10.1007/s10462-017-9547-5>.
- Agarwal, P., Mehta, S., 2014. Nature-inspired algorithms: state-of-art, problems and prospects. *Int. J. Comput. Appl.* 100, 14–21. <https://doi.org/10.5120/17593-8331>.
- Agarwal, P., Mehta, S., 2016. Enhanced flower pollination algorithm on data clustering. *Int. Journals Comput. Appl. Taylor Fr.* 7074, 144–155. <https://doi.org/10.1080/1206212X.2016.1224401>.
- Amer, A.A., Sewisy, A.A., Elgendy, T.M.A., 2017. An optimized approach for simultaneous horizontal data fragmentation and allocation in Distributed Database Systems (DDBSs). *Heliyon* 3, <https://doi.org/10.1016/j.heliyon.2017.e00487>.
- Du, J., Alhaji, R., Barker, K., 2006. Genetic algorithms based approach to database vertical partition. *J. Intell. Inf. Syst.* 26, 167–183. <https://doi.org/10.1088/1742-6596/29/1/032>.
- Goli, M., Rankoohi, S.M.T.R., 2012. A new vertical fragmentation algorithm based on ant collective behavior in distributed database systems. *Knowl. Inf. Syst.* 30, 435–455. <https://doi.org/10.1007/s10115-011-0384-6>.
- Hui Ma, K.-D., Schewe, M., Kirchberg, 2006. A heuristic approach to vertical fragmentation incorporating query information. 2006 7th Int. Balt. Conf. Databases Inf. Syst., 69–76 <https://doi.org/10.1109/DBIS.2006.1678477>.
- Khan, S.I., Hoque, D.A.S.M.L., 2010. A new technique for database fragmentation in distributed systems. *Int. J. Comput. Appl.* 5, 20–24. <https://doi.org/10.5120/940-1318>.
- Mei, M., Xue, H., Zhong, M., Gu, Y., 2010. An improved differential evolution algorithm for TSP problem, 2010 Int. Conf. Intell. Comput. Technol. Autom. ICICTA 2010 (1), 544–547. <https://doi.org/10.1109/ICICTA.2010.461>.
- Özsu, M.T., Valduriez, P., 2011. *Principles of distributed database systems*. Springer Sci. Bus. Media.
- Pazos R., R.A., Vázquez A., G., Martínez F., J.A., Pérez O., J., 2010. Vertical fragmentation design of distributed databases considering the nonlinear nature of roundtrip response time. *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*. 6277 LNAI, 173–182. https://doi.org/10.1007/978-3-642-15390-7_18.
- Pérez, J., Pazos, R., Frausto, J., Romero, D., Cruz, L., 2000. Vertical fragmentation and allocation in distributed databases with site capacity restrictions using the Threshold Accepting Algorithm. *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*. 1793 LNAI, 75–81. https://doi.org/10.1007/10720076_7.
- Rababaah, H., 2005. *Distributed Databases Fundamentals and Research*, Adv. Database-B561. Springer, 1–15.
- Rahimi, H., Parand, F.-A., Riahi, D., 2015. Hierarchical simultaneous vertical fragmentation and allocation using modified Bond Energy Algorithm in distributed databases. *Appl. Comput. Informatics*, 127–133 <https://doi.org/10.1016/j.aci.2015.03.001>.
- Storn, R., Price, K., 1997. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *J. Glob. Optim.* 11, 341–359.
- Tosun, U., 2014. Distributed Database Design using Evolutionary Algorithms. *J. Comunication Networks*. 16, 1–6. <https://www.kaggle.com/datasets>, (n.d.).